

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №4

«Работа с одномерными массивами»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 25

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

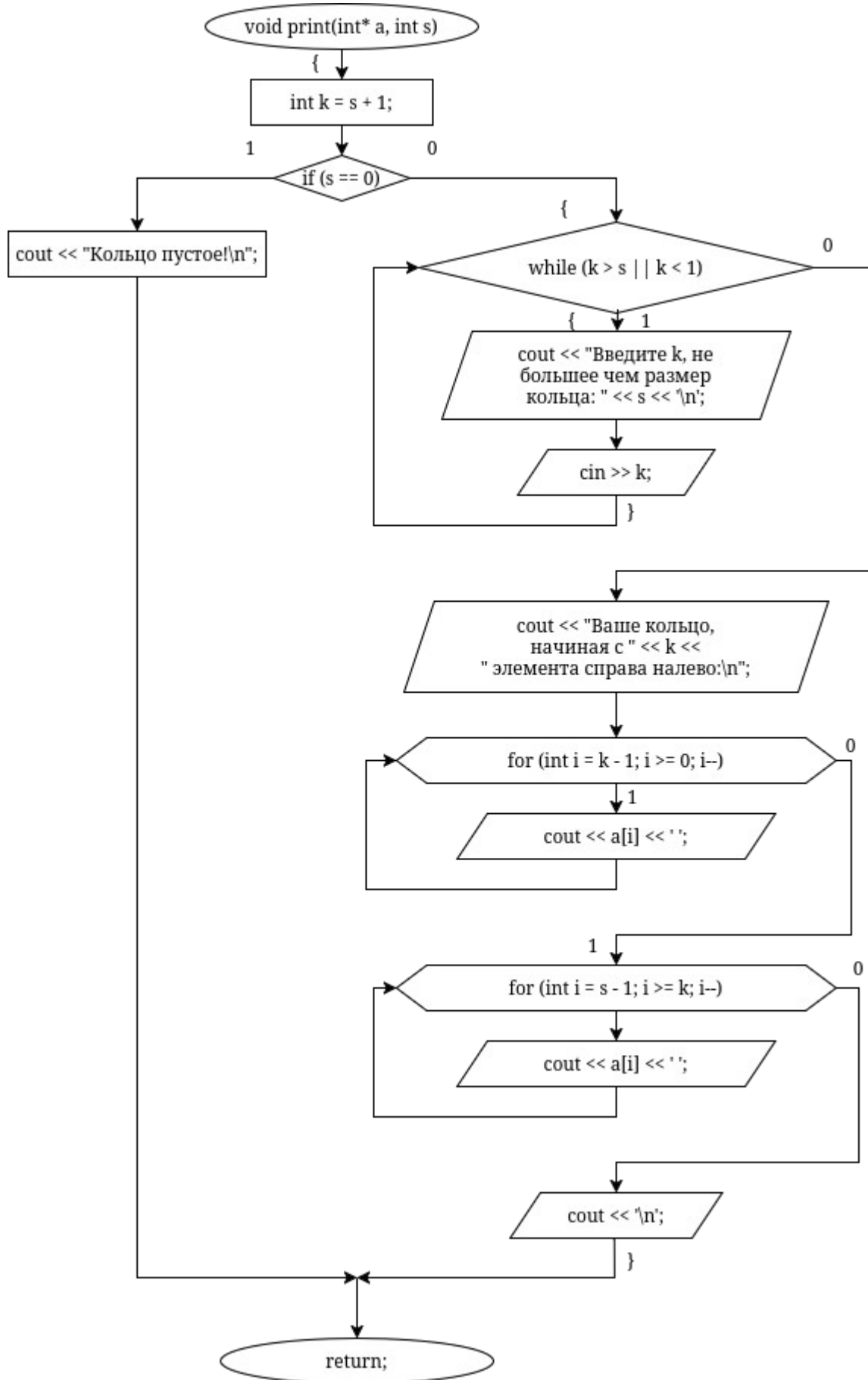
ПОСТАНОВКА ЗАДАЧИ

1. Реализовать с использованием массива однонаправленное кольцо (просмотр возможен справа налево, от первого элемента можно перейти к последнему).
2. Распечатать полученный массив, начиная с K -ого элемента и до $K+1$.
3. Упорядочить элементы по убыванию
4. Удалить из кольца нечетные элементы.
5. Распечатать полученный массив, начиная с K -ого элемента и до $K+1$.

ВЫВОД КОЛЬЦА. АНАЛИЗ

1. На вход подаётся массив и его размер. Размер внутри функции не меняется. Таким образом функция должна принимать указатель на начало массива и целое число — размер кольца (s).
2. Если размер кольца равен нулю, следует указать на это и завершить функцию.
3. Пользователю следует ввести номер элемента (k), с которого начнется вывод. Он должен быть не меньше единицы и не больше размера кольца.
4. Так как работа с кольцом идет справа налево, то сначала следует вывести элементы с $k - 1$ до 0 , идя в обратном порядке. Используем для этого `for`.
5. Далее следует вывести оставшуюся часть — цикл следует начинать с $s - 1$ до k , так же используем `for`.

БЛОК-СХЕМА



КОД

```
void print(int* a, int s) {
    int k = s + 1;
    if (s == 0) {
        cout << "Кольцо пустое!\n";
        return;
    }

    while (k > s || k < 1) {
        cout << "Введите k, не большее чем размер кольца: " << s << '\n';
        cin >> k;
    }

    cout << "Ваше кольцо, начиная с " << k << " элемента справа налево:\n";
    for (int i = k - 1; i >= 0; i--) {
        cout << a[i] << ' ';
    }
    for (int i = s - 1; i >= k; i--) {
        cout << a[i] << ' ';
    }
    cout << '\n';
}
```

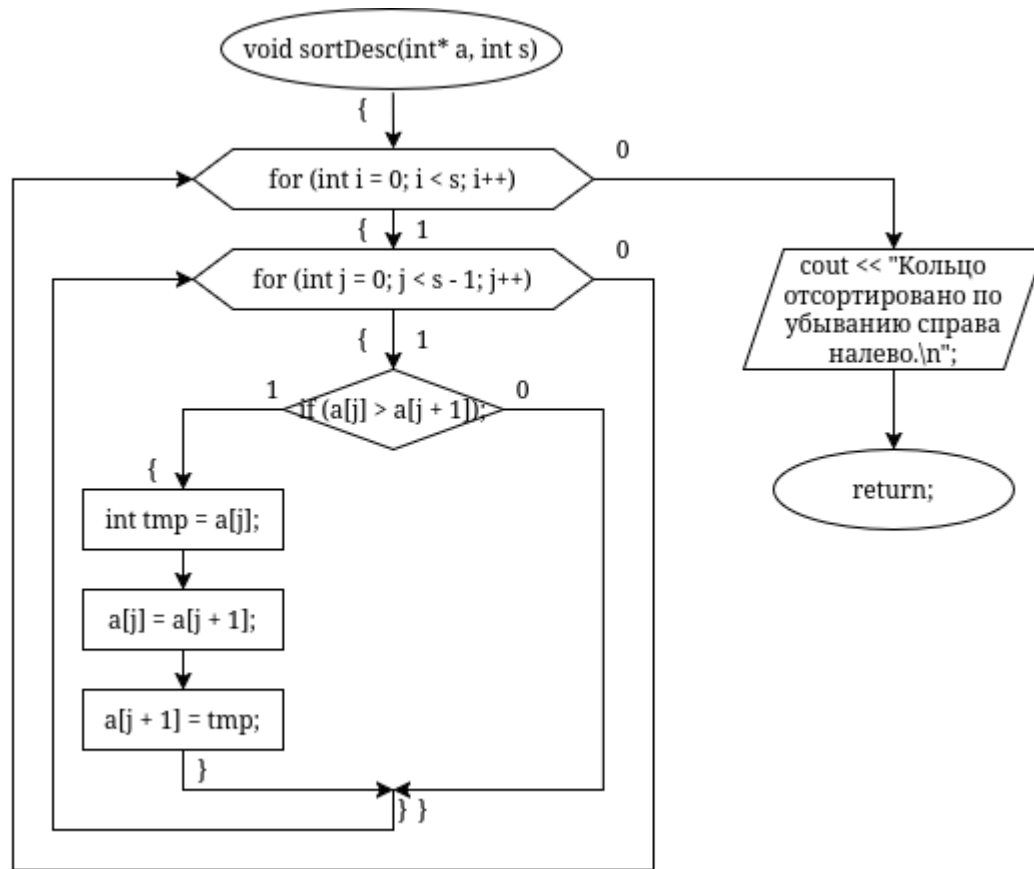
СОРТИРОВКА. АНАЛИЗ

1. На вход подаётся массив и его размер. Размер внутри функции не меняется. Таким образом функция должна принимать указатель на начало массива и целое число — размер кольца (s).

2. Используем сортировку пузырьком.

3. Так как следует отсортировать кольцо по убыванию, и первому элементу кольца соответствует последний элемент массива, то исходный массив должен быть отсортирован по возрастанию.

БЛОК-СХЕМА



КОД

```

void sortDesc(int* a, int s) {
    for (int i = 0; i < s; i++) {
        for (int j = 0; j < s - 1; j++) {
            if (a[j] > a[j + 1]) {
                int tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
    cout << "Кольцо отсортировано по убыванию справа налево.\n";
}
    
```

УДАЛЕНИЕ НЕЧЕТНЫХ ЭЛЕМЕНТОВ. АНАЛИЗ

1. На вход подаётся массив и его размер. Размер внутри функции изменяется. Таким образом функция должна принимать указатель на начало массива и указатель на целое число — размер кольца (s).

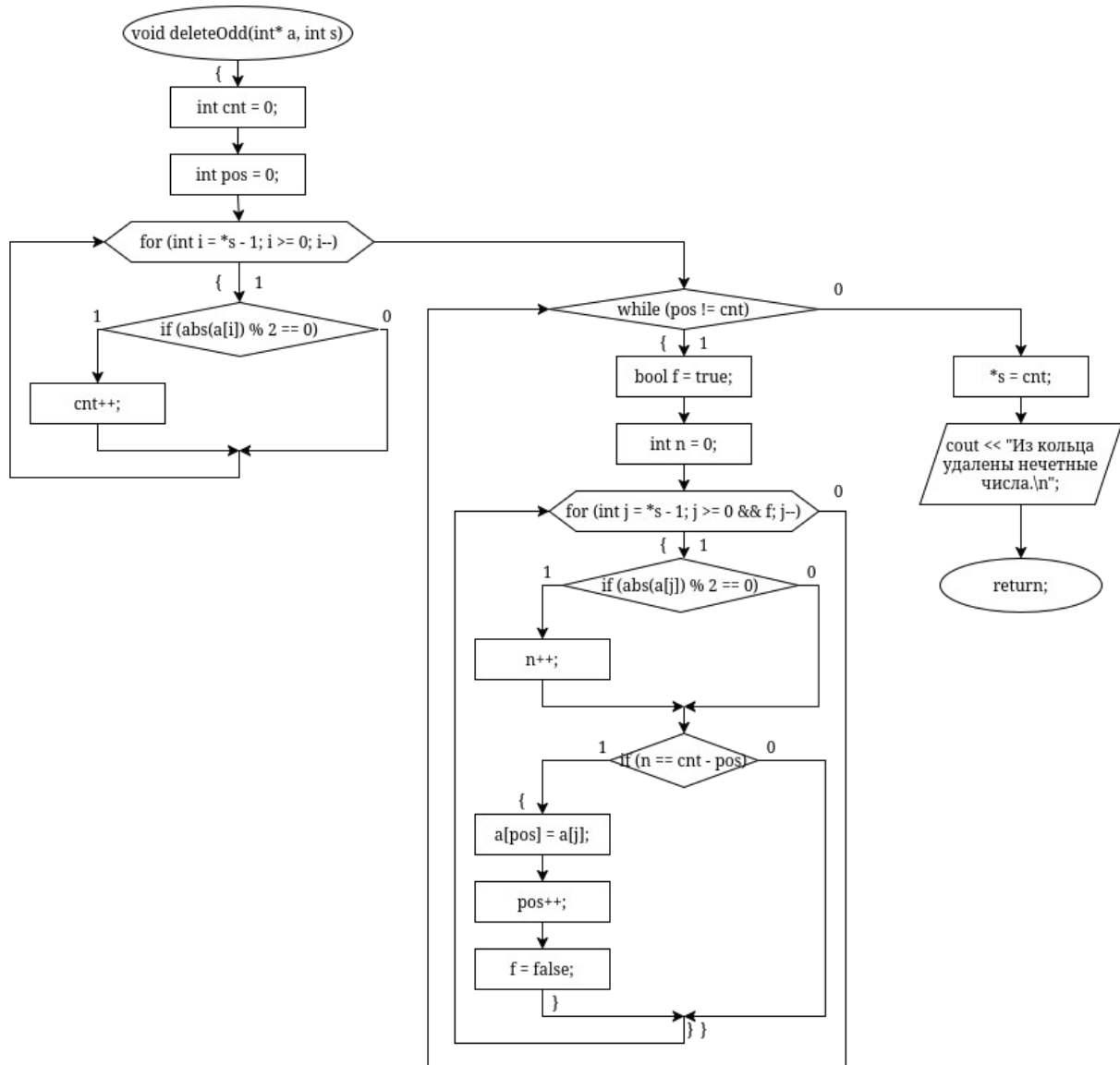
2. Подсчитам количество четных элементов в массиве. Используем для этого переменную `cnt`. Также заметим, что новым размером массива будет количество его четных элементов.

3. Требуется переменная, для позиционирования чётных чисел в массиве, назовём её `pos` и приравняем к нулю.

4. Используя цикл `for`, будем идти по направлению кольца до тех пор, пока не найдем последнее не обработанное четное число. Для этого заведем счетчик `n`. Когда `n` будет равно количеству необработанных четных чисел — `cnt - pos`, то текущий индекс станет равен индексу искомого числа. Далее перемещаем его на новую позицию `pos`, после чего увеличим эту переменную на единицу и закончим цикл. Для остановки цикла используем флаг `f`.

5. Четвертый пункт следует повторить для всех четных чисел. Заметим, что это случится, когда `pos` будет равен `cnt`.

БЛОК-СХЕМА



КОД

```
void deleteOdd(int* a, int* s) {
    int cnt = 0;
    int pos = 0;
    for (int i = *s - 1; i >= 0; i--)
        if (abs(a[i]) % 2 == 0)
            cnt++;

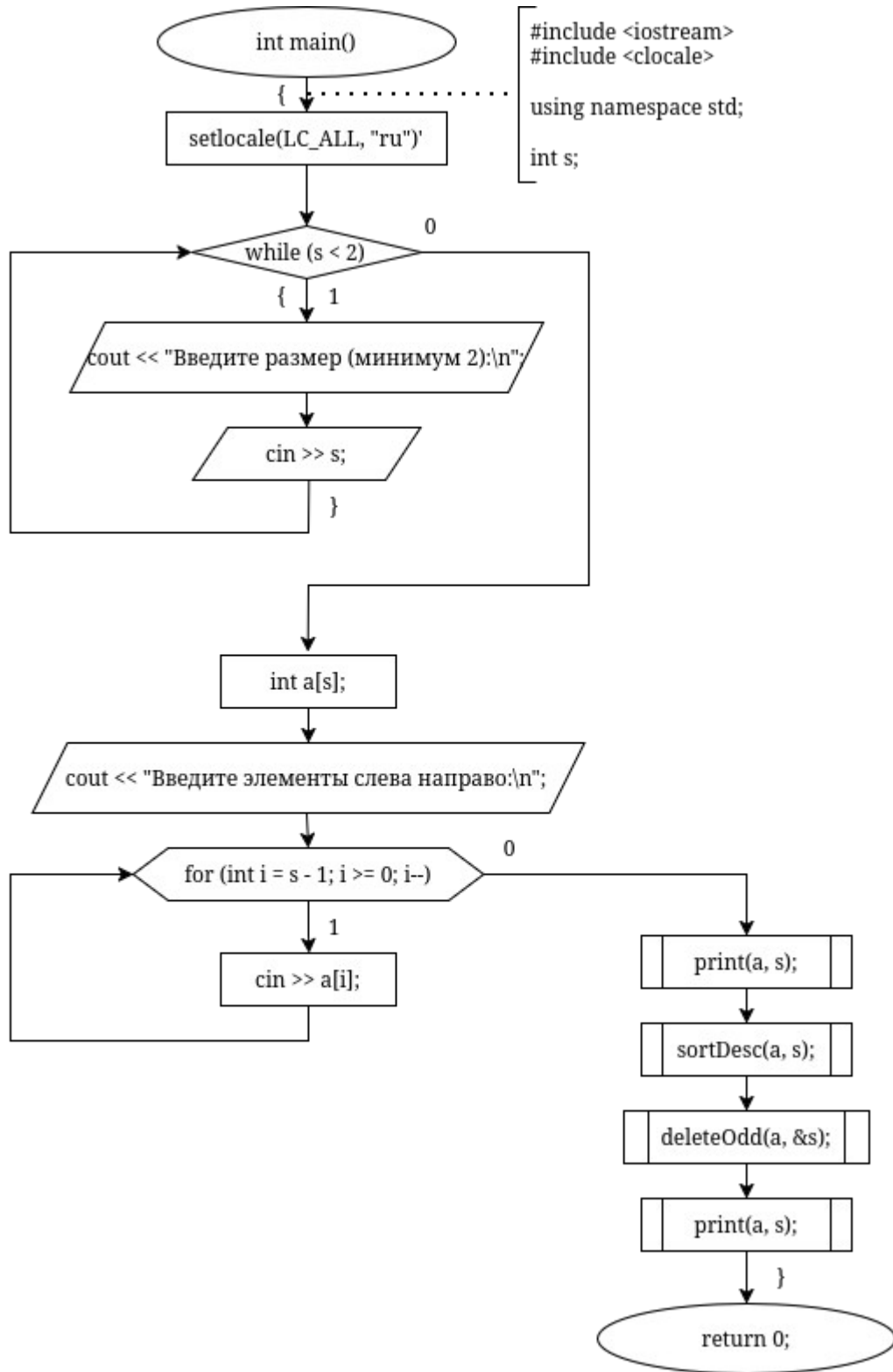
    while (pos != cnt) {
        bool f = true;
        int n = 0;
        for (int j = *s - 1; j >= 0 && f; j--) {
            if (abs(a[j]) % 2 == 0) n++;
            if (n == cnt - pos) {
                a[pos] = a[j];
                pos++;
                f = false;
            }
        }
    }

    *s = cnt;
    cout << "Из кольца удалены нечетные числа.\n";
}
```


ФУНКЦИЯ MAIN. АНАЛИЗ

1. Сперва следует ввести размер кольца, используем переменную s .
Имеет смысл ограничить его размер – минимум два элемента.
2. Далее следует ввести s элементов, начиная с начала кольца.
Используем цикл `for`, начиная с $s - 1$ и идя до нуля.
3. Далее следует вызвать функции в том порядке, в котором они указаны в постановке задачи.

БЛОК-СХЕМА



КОД

```
int main() {
    while (s < 2) {
        cout << "Введите размер (минимум 2):\n";
        cin >> s;
    }
    int a[s];
    cout << "Введите элементы слева направо:\n";
    for (int i = s - 1; i >= 0; i--)
        cin >> a[i];

    print(a, s);

    sortDesc(a, s);
    deleteOdd(a, &s);

    print(a, s);
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

```
Введите размер (минимум 2):
12
Введите элементы справа налево:
1 2 3 4 5 6 7 8 9 10 11 12
Введите k, не большее чем размер кольца: 12
6
Ваше кольцо, начиная с 6 элемента справа налево:
7 8 9 10 11 12 1 2 3 4 5 6
Кольцо отсортировано по убыванию справа налево.
Из кольца удалены нечетные числа.
Введите k, не большее чем размер кольца: 6
6
Ваше кольцо, начиная с 6 элемента справа налево:
12 10 8 6 4 2
```